

Event-Driven Pub/Sub Architecture

1. System Overview & Pub/Sub Model

To handle millions of events per minute during peak booking windows without tight coupling or cascading service degradation, BookMyShow employs an asynchronous **Publish/Subscribe (Pub/Sub)** architecture built on Apache Kafka.

Microservices communicate strictly by publishing self-contained domain events. Downstream consumer groups process these events independently at their own rate without blocking the critical booking and checkout path.

2. Event Producers, Topics, and Consumers

Event Producer	Kafka Topic	Event Type	Target Consumer Groups	Purpose
BookingService	booking-events	BookingConfirmedEvent BookingCancelledEvent	NotificationService AnalyticsService LoyaltyService	Triggers ticket delivery, updates booking dashboards, and credits reward points.
PaymentService	payment-events	PaymentCompletedEvent PaymentFailedEvent	NotificationService AnalyticsService	Sends payment receipts or failure alerts; logs transaction volumes.
ReminderService	reminder-events	ShowReminderEvent	NotificationService	Alerts users 2 hours before showtime via push notifications and SMS.
OfferService	offer-events	OfferBroadcastEvent	NotificationService AnalyticsService	Broadcasts promotional discounts and targeted marketing campaigns.

3. Standard Event Message Schema

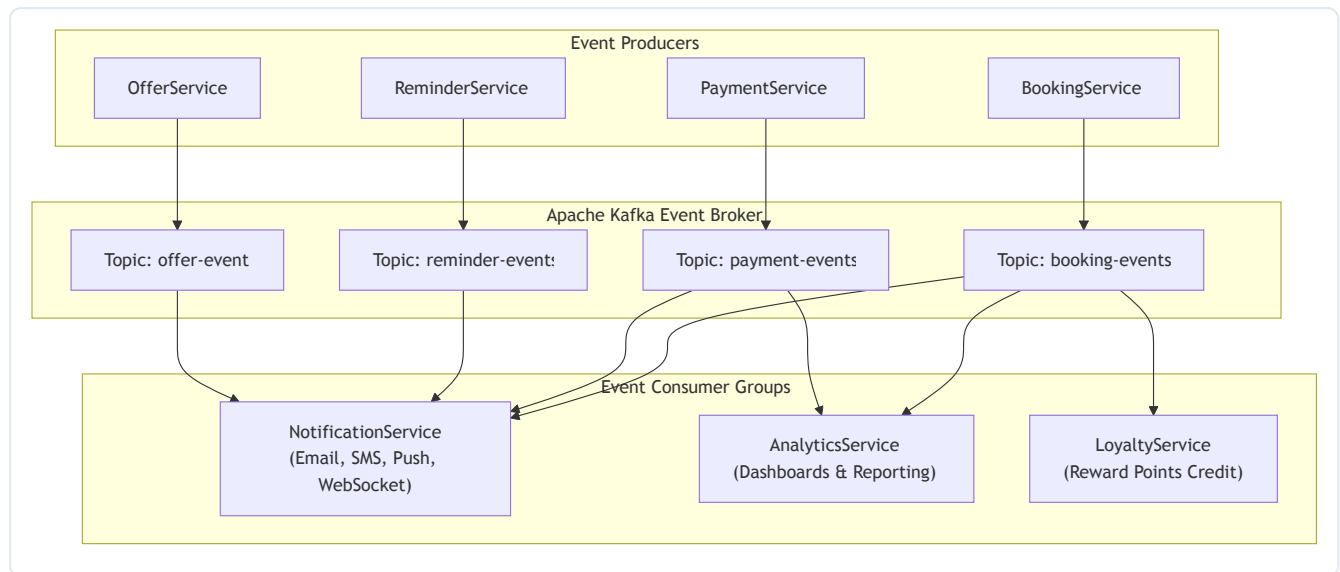
All domain events adhere to a structured JSON envelope containing event metadata, routing attributes, and domain-specific payloads:

```
{
  "eventId": "evt-12345",
  "type": "BookingConfirmedEvent",
  "timestamp": "2025-10-15T14:05:00Z",
  "data": {
    "bookingId": "BKG00123",
    "userId": "USR789",
    "showId": "SHOW55",
    "seats": ["A1", "A2"],
    "totalAmount": 680
  }
}
```

- **eventId:** Universally unique identifier used by downstream consumers for strict deduplication.
- **type:** The explicit domain event name for consumer routing and deserialization.
- **timestamp:** UTC ISO-8601 creation time used for latency tracking and SLA monitoring.
- **data:** Encapsulated event attributes required by downstream consumers without needing upstream database callbacks.

4. Pub/Sub Architecture Diagram

The architecture diagram below illustrates the end-to-end event flow across producers, the Kafka broker, and independent consumer groups:



5. Topic Partitioning & Ordering Guarantees

- **Partition Key:** All events emitted by BookingService and PaymentService use bookingId as their Kafka partition key.
- **Sequential Guarantee:** By hashing on bookingId, all related events (e.g., BookingConfirmedEvent followed by BookingCancelledEvent) land on the exact same topic partition. This guarantees that consumers process state transitions in strict chronological order.
- **Horizontal Consumer Scaling:** Each topic is divided into 12 to 24 partitions, allowing multiple worker instances within each consumer group to process incoming messages in parallel.